Árboles de Regresión

Modelos Avanzados de Aprendizaje Automático I

En el contexto del Aprendizaje Automático, los árboles de regresión constituyen una de las herramientas más intuitivas y potentes para modelar relaciones entre variables cuando la variable objetivo es continua. A diferencia de otros modelos más complejos (como redes neuronales o SVM), los árboles permiten construir modelos interpretables, basados en una secuencia de decisiones binarias sobre los atributos de entrada.

La idea fundamental consiste en particionar recursivamente el espacio de entrada en regiones más pequeñas, de manera que en cada una de ellas la variable objetivo sea lo más homogénea posible. Cada una de estas particiones se obtiene mediante condiciones del tipo:

$$x_j \leq t$$

donde x_j es una característica o variable independiente (*feature*) y t es un umbral (*threshold*). A medida que se aplican estas divisiones, se construye una estructura jerárquica en forma de árbol.

Desde un punto de vista funcional, el árbol tiene dos tipos de componentes:

- Los nodos internos representan decisiones (*splits*).
- Las hojas contienen un modelo simple. En este caso, una constante, con la media de los valores objetivo en esa región.

La predicción se realiza recorriendo el árbol desde la raíz hasta una hoja. Este enfoque se basa en un principio clave: minimizar el error dentro de cada región, que en regresión se mide habitualmente mediante el error cuadrático medio (MSE).

En este ejercicio se propone implementar un árbol de regresión desde cero en Julia. Para ello, se define una jerarquía de tipos que permite representar explícitamente la estructura del árbol.

En primer lugar, se define un tipo abstracto común:

```
abstract type ModelTree end
```

Este tipo actúa como supertipo para todos los elementos del árbol, permitiendo tratar de forma uniforme tanto nodos internos como hojas.

Los nodos internos se representan mediante la siguiente estructura:

```
mutable struct ModelTreeNode <: ModelTree
    feature    :: Int
    threshold  :: Real
    model      :: Any
    leftChild  :: ModelTree
    rightChild :: ModelTree
end
```

Como se puede ver, esta estructura define un tipo que es hijo del tipo *ModelTree*. Cada nodo contiene:

- *feature*: índice j de la característica utilizada para dividir los datos.
- *threshold*: valor umbral que define la partición.
- *model*: modelo asociado al nodo.
- *leftChild*: subárbol correspondiente a los ejemplos que cumplen $x_j \leq t$
- *rightChild*: subárbol correspondiente a los ejemplos que no cumplen la condición.

Sobre el campo *model* de esta estructura, es importante tener en cuenta dos cosas:

- Aunque el modelo final se encuentra en las hojas, almacenar un modelo en cada nodo puede resultar útil (por ejemplo, para hacer poda o suavizado de las predicciones).
- En este caso, para Árboles de Regresión, el modelo será la media de los *targets* en ese nodo, es decir, un valor numérico. Como se puede ver, el tipo de *model* es *Any* y no *Real*. Esto se ha definido así porque esta estructura se utilizará en el siguiente ejercicio para almacenar modelos de regresión de cualquier tipo.

Las hojas del árbol se representan mediante la siguiente estructura, que define un tipo que también es hijo de *ModelTree*:

```
struct ModelTreeLeaf <: ModelTree
    model      :: Any
    n          :: Int
end
```

Donde:

- *model*: valor de predicción en esa región. Igual que antes, para el caso de Árboles de Regresión debería ser de tipo *Real*, pero se hace de tipo *Any* para que, en el siguiente

ejercicio, se pueda almacenar cualquier tipo de modelo.

- n: número de instancias de entrenamiento que han llegado a esa hoja.

Las hojas representan las regiones finales del espacio de entrada, donde ya no se realizan más divisiones. Por tanto, constituyen el punto final del proceso recursivo de construcción del árbol.

Para evaluar un árbol, se hace descender una instancia hasta una hoja, y se devuelve el modelo (valor real) que contiene esa hoja. Para hacerlo descender en un nodo, se toma el valor j del campo *feature* y el umbral t del campo *threshold*, y, si $x_j \leq t$, se hace descender por el hijo izquierdo; en caso contrario, por el hijo derecho.

Para generar un árbol, se hace de la siguiente manera:

1. Se parte de todo el conjunto de datos (raíz).
2. Se crea un modelo con el valor promedio de las salidas deseadas de ese conjunto de datos.
3. Si se cumple alguna condición de parada (profundidad máxima o número mínimo de muestras), se devuelve una hoja con este modelo.
4. Si no:
    1. Se busca la mejor división posible (*feature* + *threshold*).
    2. Se separan los datos en dos subconjuntos mediante $x_j \leq t$.
    3. Se repite el proceso de forma recursiva en cada subconjunto, obteniendo 2 árboles.
    4. Se asignan ambos árboles como hijos izquierdo y derecho.

Este procedimiento permite construir modelos no lineales complejos a partir de reglas simples, lo que hace que los árboles de regresión sean una herramienta fundamental dentro de los modelos basados en particiones del espacio.

En este ejercicio, se pide realizar las siguientes funciones:

- *mse*. En problemas de regresión, es necesario disponer de una métrica que permita cuantificar la diferencia entre los valores predichos por el modelo y los valores reales (*targets*). Una de las métricas más utilizadas es el error cuadrático medio (*Mean Squared Error*, MSE), debido a su simplicidad y a sus buenas propiedades analíticas. El MSE mide el promedio de los errores al cuadrado entre las predicciones y los valores reales. Penaliza en mayor medida los errores grandes, lo que resulta especialmente útil en muchos problemas de regresión donde se desea evitar desviaciones significativas. Esta función recibe como parámetros:
    - *outputs*, de tipo *Union{Real,AbstractVector{<:Real}}*, con las salidas de un modelo.

Estas salidas podrán ser de tipo *Real*, con el mismo valor para todas las instancias, o ser un vector con un valor distinto para cada instancia.

- *targets*, de tipo *AbstractVector{<:Real}*, con las salidas deseadas. En este caso, solamente puede ser un vector.

Esta función deberá devolver un único valor numérico, con el MSE calculado a partir de *outputs* y *targets*. El valor que devuelva deberá ser del mismo tipo que los datos de *outputs* y *targets*. Para el desarrollo de esta función no se permite el uso de bucles.

- *candidateThresholds*. En la construcción de un árbol de regresión, uno de los pasos fundamentales consiste en determinar dónde realizar las divisiones (*splits*) sobre cada característica. Para ello, es necesario definir un conjunto de posibles valores de umbral (*thresholds*) para cada atributo que serán evaluados durante el proceso de búsqueda del mejor *split*. Esta es la tarea que realiza esta función. Recibe un único parámetro:
  - *x*, de tipo *AbstractVector{<:Real}*, con los distintos valores de este atributo en el conjunto de entrenamiento.

  Esta función debe devolver un vector de elementos del mismo tipo que los elementos de *x* con los umbrales. Además, este vector deberá estar ordenado de menor a mayor. Para hacer esto, se tomarán en primer lugar los valores únicos (función *unique*) de *x*, se ordenan, y, para cada par de valores contiguos de este nuevo vector, se toma el valor promedio. Si el vector de valores únicos solamente tiene un valor, devolverá un vector vacío, con los elementos definidos del mismo tipo que los elementos de *x*.
  Para el desarrollo de esta función no se permite el uso de bucles.

- *splitDataset*. Esta función permite dividir una matriz en dos subconjuntos dado un atributo y un umbral. Esta función recibe los siguientes parámetros:
  - *inputs*, de tipo *AbstractMatrix{<:Real}*, con la matriz a dividir, con las instancias en las filas y los atributos en las columnas.
  - *numFeature*, de tipo *Int*, con el índice del atributo utilizado para partir el *dataset*.
  - *threshold*, de tipo *Real*, con el valor del umbral en ese atributo.

  Esta función deberá devolver una tupla con tres elementos, los dos primeros del mismo tipo que *inputs* (y cuyos elementos deberán ser del mismo tipo que los de *inputs*), con los dos subconjuntos. El primero contendrá los datos para los cuales el valor del atributo *numFeature* es menor o igual que *threshold*, y el segundo los restantes datos, manteniendo ambos el mismo orden que en el conjunto original. El tercera valor de la tupla será los índices de los elementos que se corresponden al primer subconjunto, como un vector de

valores booleanos de longitud igual al número de datos de *inputs* (número de filas). Para el desarrollo de esta función no se permite el uso de bucles.

- *splitDataset*. Función anterior sobrecargada, que, en este caso, permite dividir un *dataset* de tipo *Batch* dos subconjuntos dado un atributo y un umbral. Recibe los siguientes parámetros:
    - *dataset*, de tipo *Batch*, con el conjunto de datos a dividir.
    - *numFeature*, de tipo *Int*, con el índice del atributo utilizado para partir el *dataset*.
    - *threshold*, de tipo *Real*, con el valor del umbral en ese atributo.

  De forma análoga a la anterior, esta función deberá devolver una tupla con tres elementos, siendo los dos primero de tipo *Batch*, con los dos subconjuntos. El primero contendrá los datos para los cuales el valor del atributo *numFeature* es menor o igual que *threshold*, y el segundo los restantes datos, manteniendo ambos el mismo orden que en el conjunto original. Al igual que en la anterior, el tercer elemento será un vector de valores booleanos indicando los elementos del *dataset* tomados para el primer subconjunto. Esta función se puede hacer fácilmente llamando a la anterior, y usando la función *selectInstances* desarrollada previamente. Para el desarrollo de esta función no se permite el uso de bucles.

- *calculateErrorSplit*. Una vez definidos los posibles umbrales para cada característica, el siguiente paso en la construcción del árbol de regresión consiste en evaluar la calidad de cada posible división. Para ello, es necesario medir el error que se obtiene al particionar el conjunto de datos según un determinado *feature* y un *threshold*, tarea que realiza esta función. Recibe como parámetros:
    - *dataset*, de tipo *Batch*, con el conjunto de entrenamiento.
    - *numFeature*, de tipo *Int*, con el índice del atributo usado para partir el *dataset*.
    - *threshold*, de tipo *Real*, con el umbral usado en el atributo anterior para partir el *dataset*.

  Esta función debe dividir el *dataset*, pasado como parámetro, en dos partes según el atributo de índice *numFeature* j y el *threshold* t, haciendo, como se ha mostrado anteriormente, $x_j \leq t$. Para ello, se puede usar la función anterior *splitDataset*. Una vez se tiene cada parte, se toma el valor promedio de la salida deseada (función *batchTargets*) de cada una, y se calcula el MSE con la función anterior, usando como *outputs* ese valor promedio, y las propias salidas deseadas de cada partición. Finalmente, se devolverá la suma de estos dos valores de error, ponderada por el número de instancias de cada subconjunto. El valor que devuelva deberá ser del mismo tipo que los datos del *dataset*.

Para el desarrollo de esta función no se permite el uso de bucles.

- *bestSplit*. Una vez que se dispone de un mecanismo para generar umbrales candidatos y evaluar el error de cada posible división, el siguiente paso consiste en seleccionar la mejor partición posible para un conjunto de datos dado. Este proceso se implementa mediante esta función, que recibe un único parámetro:
    - *dataset*, de tipo *Batch*, con el conjunto de entrenamiento.

  Esta función devolverá la mejor combinación de característica (*feature*) y umbral (*threshold*) que minimiza el error de partición como una tupla (tipo *Tuple*) con dos valores, el primero de tipo *Int* (con el índice del atributo), y el segundo del mismo tipo que los elementos de la matriz de entradas del *dataset*, con el valor de umbral.

  Para desarrollar esta función, en primer lugar se comprobará que el *dataset* tenga más de una instancia. Si no es así, no se podrá calcular el umbral, y se devolverá la tupla cuyo primer valor es -1, y el segundo es *NaN*, de los tipos anteriormente especificados.

  Si hay más de un elemento, se deberá calcular el error del conjunto de datos pasado como argumento, mediante la función *mse* usando como *outputs* el promedio de las salidas deseadas, y multiplicar este por el número de instancias de este conjunto de entrenamiento. Después, se creará un bucle que recorra los atributos y, para cada atributo, otro bucle que recorra los posibles umbrales (calculado por medio de la función *candidateThresholds*). En el cuerpo de este bucle doble se llamará a la función *calculateErrorSplit* para calcular el error de este umbral con este atributo. Finalmente, se devolverá una tupla con los tipos especificados, donde el primer elemento será el índice del atributo y el segundo el umbral que devuelven el menor error. Si este valor del menor error fuese el calculado antes de los bucles, la tupla deberá tener como primer valor -1, y como segundo valor *NaN*, con los tipos antes especificados.

  Para el desarrollo de esta función se permite hacer dos bucles, para recorrer los atributos, y, interno a este, para recorrer los umbrales para cada atributo.

- *buildRegressionTree*. Una vez definidos todos los componentes necesarios (métrica de error, generación de umbrales y búsqueda del mejor *split*), el siguiente paso consiste en implementar el proceso completo de construcción del árbol de regresión. Este proceso se realiza de forma recursiva mediante esta función, que recibe un único argumento:
    - *dataset*, de tipo *Batch*, con el conjunto de entrenamiento.

  Además, esta función recibe como argumentos opcionales los hiperparámetros de este algoritmo contemplados en este ejercicio:

- *max_depth*, de tipo *Int*, que contiene la profundidad máxima del árbol a generar. Su valor por defecto es -1, con lo que se indica que se generarán árboles de cualquier profundidad. Funciona como criterio para parar la recursividad.
- *min_samples_leaf*, de tipo *Int*, que contiene el número mínimo de instancias del conjunto de entrenamiento que debe tener cada nodo. Al igual que el anterior, funciona como criterio parar parar el proceso recursivo. Su valor por defecto es 1.

Esta función devolverá un Árbol de Regresión, de tipo *ModelTree*, aplicando un proceso recursivo en el que el caso base es el siguiente: Si el número de muestras del conjunto pasado como argumento es inferior o igual a *min_samples_leaf*, o bien si la profundidad máxima es (parámetro *max_depth*) es 0 (la profundidad de un nodo es el número de divisiones o *splits* desde la raíz hasta ese nodo, y, por tanto, la raíz tiene una profundidad de 0), entonces se creará y se devolverá un nodo hoja, llamando al tipo *ModelTreeLeaf* como si fuese una función, especificando como argumentos los valores por orden de cada uno de los campos, en este caso, el modelo que contiene la hoja y el número de instancias. Para el caso de un Árbol de Regresión, el modelo será el valor promedio de las salidas deseadas del conjunto pasado como argumento.

Si no se cumple el caso base, será necesario dividir el *dataset*, para lo cual se llamará a la función *bestSplit*. Como resultado de esta llamada se tendrá el índice del atributo y el valor del umbral para hacer la partición. Si este índice es -1, quiere decir que no se ha encontrado una partición que genere subconjuntos cuyos errores combinados sean mejores que el error actual. En este caso, se devolverá una hoja de la misma manera que en el caso base de la recursividad.

Una vez se tiene el índice j y el umbral t para dividir el *dataset*, se puede utilizar la función *splitDataset*, y se hacen dos llamadas recursivas con los dos subconjuntos resultantes, el primero para el hijo izquierdo y el segundo para el derecho, teniendo como resultado dos árboles, de tipo *ModelTree*. Para hacer estas dos llamadas es importante tener en cuenta que en cada una habrá que pasar los siguientes argumentos:

- *dataset*: el subconjunto correspondiente.
- *max_depth* (argumento opcional): el mismo valor que ha recibido esta llamada, pero restándole 1.
- *min_samples_leaf* (argumento opcional): el mismo valor que ha recibido esta llamada.

Finalmente, se devolverá un nodo creado mediante el uso del tipo *ModelTreeNode* como función, pasándole los valores de los campos como argumentos, por el mismo orden. Al igual que antes, el modelo será el valor promedio de todas las salidas deseadas del *dataset*

pasado como argumento. Para el desarrollo de esta función no se permite el uso de bucles.

- *predict*. Una vez construido el árbol de regresión, es necesario definir cómo utilizarlo para realizar predicciones sobre nuevos datos. En estos ejercicios se definirá una interfaz similar a la usada en la librería MLJ, basada en el uso de la función *predict*, que permite obtener las salidas del modelo a partir de una matriz de entradas. En estos ejercicios, además, se trabajará con varios tipos de modelos distintos, por lo que se deberá definir la función *predict* para cada modelo distinto. Además, se tomará una definición más amplia de lo que es un modelo. Por ejemplo, en árboles de regresión un modelo será tanto el propio árbol como el valor escalar que almacenan los nodos y las hojas de los árboles. Todas las versiones sobrecargadas (métodos) de *predict*, tanto en este ejercicio como en los siguientes, recibirán los mismos argumentos:
    - *model*, será del tipo determinado por el modelo del cual se quiere definir la función de predicción, y contiene el propio modelo.
    - *inputs*, de tipo *AbstractMatrix{<:Real}*, con una matriz de datos para hacer las predicciones, donde cada dato se sitúa en una fila distinta y las columnas son los atributos.

  En este ejercicio, se definirá la función *predict* para que reciba como modelo un árbol de regresión, o bien un valor escalar. En todos los casos, esta función deberá devolver un vector de valores reales, con un valor de predicción por cada fila del argumento *inputs*. Si bien lo idóneo sería que el tipo de estos valores fuera del mismo que el tipo de los elementos de *inputs*, esto no se puede asegurar, puesto que ya depende de los modelos con los que se trabaje y los tipos internos que estos utilicen en sus parámetros. Para hacer esta función, se definirán 3 métodos o funciones sobrecargadas, variando únicamente el tipo del argumento *model*:
    - Si es de tipo *Real*, el modelo será un valor escalar, indicando que deberá devolver ese valor para todas las instancias del argumento *inputs*. Para hacer esto, es útil utilizar la función *fill*.
    - Si es de tipo *ModelTreeLeaf*, el modelo será una hoja de un Árbol de Regresión. En este caso, se toma el modelo contenido en la hoja (en el campo *model*), y se utiliza para hacer la predicción. Si bien en este ejercicio este modelo interno será un valor escalar y esta operación se puede hacer de forma similar a la anterior, esta operación se deberá realizar mediante una llamada a la función *predict* pasando como primer argumento el modelo interno de la hoja, y como segundo el conjunto de entradas *inputs*. Esto se hace así para posibilitar, en el siguiente

ejercicio, que las hojas contengan modelos distintos a un valor escalar.

- Si es de tipo *ModelTreeNode*, el modelo será un nodo interno de un Árbol de Regresión. En este caso será necesario partir el conjunto de entradas según el índice j del atributo y el umbral almacenados en el propio nodo de la forma que se ha venido utilizando en este ejercicio ($x_j \leq t$ y $x_j > t$), hacer llamadas recursivas para ambos hijos con cada subconjunto ($x_j \leq t$ para el hijo izquierdo y $x_j > t$ para el hijo derecho), y ensamblar los resultados en el vector final.

Para el desarrollo de estas funciones sobrecargadas no se permite el uso de bucles.

Firmas de las funciones:

A continuación se muestran las firmas de las funciones a realizar en los ejercicios propuestos. Tened en cuenta que, dependiendo de cómo se defina la función, esta puede contener o no la palabra reservada *function* al principio:

```
function mse(outputs::Union{Real,AbstractVector{<:Real}},
             targets::AbstractVector{<:Real})

function candidateThresholds(x::AbstractVector{<:Real})

function splitDataset(inputs::AbstractMatrix{<:Real}, numFeature::Int,
            threshold::Real)
function splitDataset(dataset::Batch, numFeature::Int, threshold::Real)

function calculateErrorSplit(dataset::Batch, numFeature::Int, threshold::Real)

function bestSplit(dataset::Batch)

function buildRegressionTree(dataset::Batch;
    max_depth        :: Int = -1,
    min_samples_leaf :: Int =  1
)

function predict(model::Real,         inputs::AbstractMatrix{<:Real})
function predict(tree::ModelTreeLeaf, inputs::AbstractMatrix{<:Real})
function predict(tree::ModelTreeNode, inputs::AbstractMatrix{<:Real})
```